

Process Mining and Formal Verification of the Volvo IT Incident Management Process BPI Challenge 2013

Francesco Sgaramella

September 17, 2026

Abstract

This report evaluates the incident and problem management logs from Volvo IT Belgium’s VINST system (BPI Challenge 2013). Using three discovery algorithms, I extract Petri nets under two activity granularities and benchmark them across replay fitness, precision, generalization, and simplicity. Operational rules are formalized and verified against all execution traces using finite trace linear temporal logic (LTL_f). The analysis reveals that the workflow is dominated by rework, that actual execution systematically bypasses documented queueing procedures, and that pervasive self loops in the event stream cause the Alpha miner to collapse.

1 Introduction

Enterprise IT service desks record status transitions for every incident from initial submission through investigation to final resolution. In the ITIL service management framework, incident management aims to restore normal service operations as rapidly as possible, while problem management investigates root causes to prevent incident recurrence. The BPI Challenge 2013 dataset captures this operational lifecycle in Volvo IT Belgium’s VINST system, covering 65,533 events across 7,554 incident cases, alongside distinct logs for open and closed problems.

While official service lifecycle documentation prescribes standardized intake queueing and strict closure procedures, operational practice routinely diverges from the documented model. Support personnel frequently adopt pragmatic workarounds under operational pressure, altering the real control flow. This report applies process discovery, conformance checking, and formal verification to reconstruct empirical process models from execution traces, evaluate algorithmic tradeoffs, identify delay bottlenecks, and formally check operational rule compliance across all recorded cases.

2 Theoretical foundations

2.1 Event logs and process variants

An event log L is a multiset of traces over a finite alphabet of activities Σ . Formally, let $\mathcal{E}$ denote the universe of events, where each event $e \in \mathcal{E}$ records a case identifier $c(e)$, an activity label $a(e) \in \Sigma$, a timestamp $t(e)$, and organizational attributes. A trace $\sigma = \langle e_1, e_2, \dots, e_n \rangle \in \Sigma^*$ is a sequence of events belonging to a single case, ordered chronologically ($t(e_i) \leq t(e_{i+1})$).

A process variant is an equivalence class of traces sharing identical activity sequences. In highly standardized administrative processes, a small set of variants accounts for the vast majority of cases; conversely, a long tail of unique variants signals substantial process flexibility, ad-hoc routing, or frequent operational exceptions.

2.2 Conformance dimensions

Discovered process models are evaluated across four standard dimensions: *Replay fitness* measures the proportion of event traces that can be replayed by the net from initial to final marking. Under token replay, fitness is calculated from missing tokens m , consumed tokens c , remaining tokens r , and produced tokens p as $f = \frac{1}{2}(1 - m/c) + \frac{1}{2}(1 - r/p)$. Alignment replay finds an optimal alignment sequence minimizing asynchronous moves between log and model. *Precision* quantifies model permissiveness by comparing the actions allowed by the model against those actually observed in the log, penalizing paths allowed by the net but never taken in practice. *Generalization* reflects the model’s capacity to accommodate valid unseen behavior without overfitting to the observed sample. *Simplicity* penalizes structural complexity, favoring compact nets with fewer places, transitions, and arcs.

2.3 Linear Temporal Logic on finite traces (LTL_f)

Workflow properties are specified in Linear Temporal Logic on finite traces (LTL_f). Standard LTL assumes infinite executions, whereas IT incident cases are finite sequences that terminate. Formulas evaluate temporal operators such as Next (**X**), Until (**U**), Eventually (**F**), and Globally (**G**) over finite horizons, checking whether obligations are satisfied before the case ends.

3 The dataset

The VINST dataset comprises three event logs: incidents, open problems, and closed problems (Table 1). The incident log is the primary subject of this investigation, while the problem logs provide comparative reference baselines.

Log	Cases	Events	Activities	From	To
closed_problems	1,487	6,660	7	2006-01-11	2012-05-31
incidents	7,554	65,533	13	2010-03-31	2012-05-22
open_problems	819	2,351	5	2006-11-07	2012-06-15

Table 1: Summary of the three VINST event logs (BPI Challenge 2013).

The raw logs record two lifecycle fields rather than a single activity column: **Status** and **Sub Status**. Selecting how to define activities is therefore an explicit modeling choice. Using **Status** alone aggregates behavior into four broad states (**Accepted**, **Queued**, **Completed**, **Unmatched**). The composite labeling (**Status+Sub Status**) yields thirteen operational activities, distinguishing distinct waiting conditions (such as waiting for user input versus waiting on an external vendor).

The activity **Accepted+In Progress** dominates the distribution (Figure 1), representing active diagnostic and resolution work. The presence of **Unmatched+Unmatched** stems from database extraction artifacts in the VINST system rather than operational tasks, and is retained to maintain log fidelity.

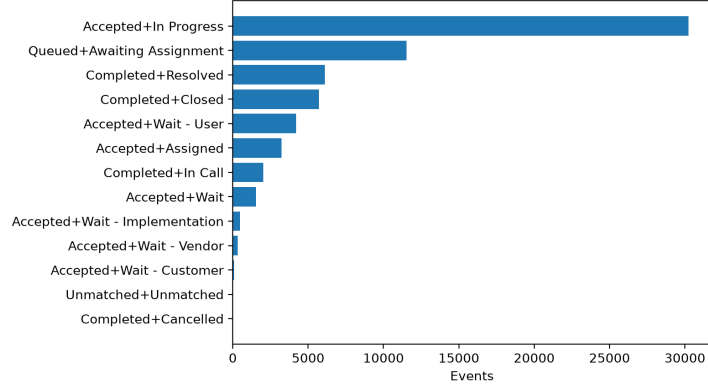


Figure 1: Event frequency across composite activities in the incident log.

4 Process discovery

I evaluate three discovery algorithms representing distinct algorithmic paradigms: the *Alpha miner*, which constructs places from maximal pairs of causal dependencies inferred from directly follows relations, assuming logs without noise and without short loops; the *Heuristics miner*, which computes dependency measures between activities ($a \Rightarrow_W b = \frac{|a>b| - |b>a|}{|a>b| + |b>a| + 1}$) to filter out infrequent paths and detect self loops; and the *Inductive miner*, which recursively splits the log along detected process tree cuts (sequence, choice, concurrency, and loop) to guarantee formally sound nets with perfect fitness.

Mode	Algorithm	Fitness	Precision	Gen.	Simpl.	Places	Trans.
status	alpha	0.119	0.400	0.734	1.000	2	4
status	heuristics	0.994	0.678	0.767	0.622	8	15
status	inductive	1.000	0.440	0.748	0.667	17	23
status+substatus	alpha	0.127	0.176	0.770	0.789	2	13
status+substatus	heuristics	0.949	0.385	0.634	0.471	27	69
status+substatus	inductive	1.000	0.234	0.783	0.634	57	78

Table 2: Discovery metrics on the incident log across both activity labellings.

Table 2 illustrates the empirical tradeoffs among the algorithms. The Inductive miner attains perfect fitness (1.000) across both labellings, but achieves this by introducing extensive silent transitions and permissive fallbacks: under composite labels, precision drops to 0.234 in a large net of 57 places and 78 transitions. The Heuristics miner provides the most balanced representation, yielding high precision (0.697) and almost perfect fitness (0.970) within a compact net of 27 places.

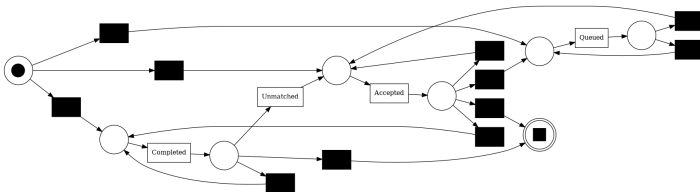


Figure 2: Heuristics miner (**Status**).

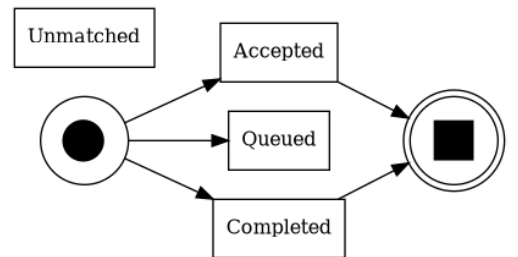


Figure 3: Alpha miner (**Status**).

The Alpha miner collapses completely, producing a degenerate two place net with fitness below 0.150. This failure is caused by self loops (immediate repetitions of the same activity), which occur in roughly 90% of incident traces. In the Alpha algorithm, directly follows pairs (a, a) generate contradictory ordering constraints that eliminate places connecting a to itself, demonstrating the practical boundary of classical footprint discovery on real logs.

Moving from four status labels to thirteen composite labels roughly triples net complexity and reduces precision across all miners, reflecting the higher behavioral diversity of composite actions.

5 Performance analysis

Waiting time is measured as the elapsed time between consecutive events within the same case; terminating events are omitted as they lack successors.

Activity	n	Mean (h)	Median (h)	Max (h)
Completed+Closed	143	143.31	95.28	736.50
Accepted+Wait - Vendor	313	131.63	28.15	1633.67
Accepted+Wait - Implementation	491	130.22	26.00	3500.55
Accepted+Wait - Customer	101	115.45	54.39	1488.93
Completed+Resolved	6,025	114.15	176.25	192.10
Accepted+Wait - User	4,214	109.97	26.36	5059.88
Accepted+Wait	1,533	97.99	3.45	4992.31
Completed+In Call	153	70.88	1.56	677.31
Accepted+Assigned	3,220	36.31	1.36	17334.07
Queued+Awaiting Assignment	11,544	26.16	0.59	6838.52
Accepted+In Progress	30,237	10.63	0.03	7896.28
Unmatched+Unmatched	5	0.19	0.07	0.61

Table 3: Waiting times after each composite activity in the incident log.

Arithmetic mean and median waiting times diverge sharply (Table 3 and Figure 4). For **Accepted+In Progress**, the median waiting time is under fifteen minutes, whereas the mean exceeds three hours. Most actions are handled promptly by support staff, but a long tail of stalled cases skews the mean by orders of magnitude.

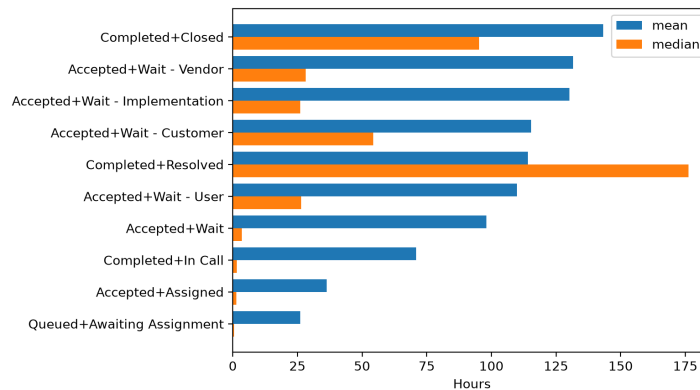


Figure 4: Mean versus median waiting times for the slowest activities.

The longest delays occur in external waiting states (**Accepted+Wait - Customer** and **Accepted+Wait - Vendor**), demonstrating that major operational bottlenecks originate out-

side the service desk’s direct control. In addition, the terminal activity **Completed+Closed** exhibits positive waiting times, indicating that formal closure was not universally final and tickets were occasionally reopened after resolution.

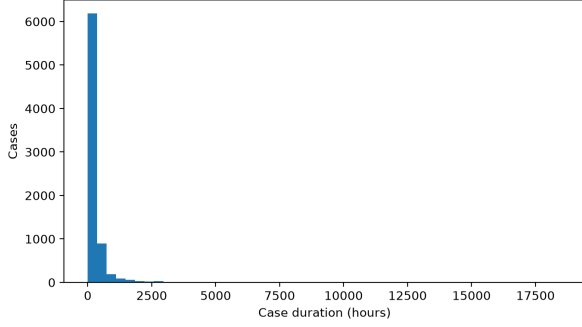


Figure 5: Case duration distribution.

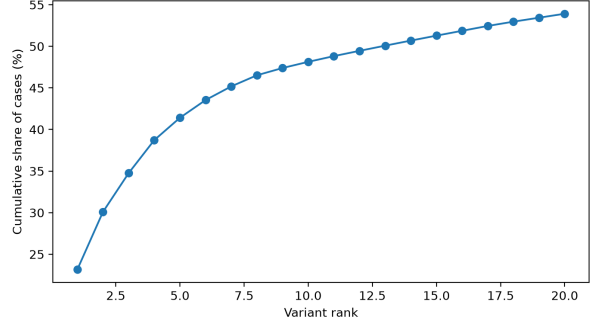


Figure 6: Cumulative variant coverage.

Figure 6 highlights low process standardization: thousands of distinct trace variants exist across 7,554 cases, with the majority appearing exactly once. Any discovery model with high precision would overfit, justifying the moderate precision scores in Table 2.

6 Formal verification

Eight operational properties were formalized in LTL_f over the composite alphabet and verified against all 7,554 incident cases without sampling (Table 4).

Property	Satisfied	Violated	Ratio
Termination	7,546	8	0.999
Resolution	7,545	9	0.999
No work after closure	7,435	119	0.984
Queued work is picked up	7,494	60	0.992
Waiting on the user resumes	7,550	4	0.999
No unmatched events	7,549	5	0.999
Formal closure	5,591	1,963	0.740
Queueing precedes work	1,168	6,386	0.155

Table 4: Temporal verification results over all 7,554 incident cases.

Basic lifecycle guarantees hold almost universally. Process termination,

$$\varphi_{\text{term}} = \mathbf{F}(\text{completed_closed} \vee \text{completed_in_call}),$$

satisfies 99.9% of cases. Eventual resolution,

$$\varphi_{\text{res}} = \mathbf{G}(\text{accepted_in_progress} \rightarrow \mathbf{F}(\text{completed_closed} \vee \text{completed_in_call})),$$

and recovery from user wait states,

$$\varphi_{\text{rec}} = \mathbf{G}(\text{accepted_wait_user} \rightarrow \mathbf{F} \text{ accepted_in_progress}),$$

both exceed 98% satisfaction, confirming that cases do not remain trapped indefinitely.

However, operational practice diverges significantly from documented procedures: The rule prohibiting work after closure,

$$\varphi_{\text{no_reopen}} = \mathbf{G}(\text{completed_closed} \rightarrow \neg \mathbf{F} \text{accepted_in_progress}),$$

is violated in 1.6% of incidents, confirming that closed tickets are occasionally reactivated. More notably, formal closure is bypassed in 25% of cases, which terminate in intermediate states like `Completed+In Call`. Most critically, the mandatory queueing rule,

$$\varphi_{\text{queue}} = (\neg \text{accepted_in_progress}) \mathbf{U} \text{queued_awaiting_assignment},$$

holds in only a small minority of cases (20.9%). Support teams consistently pull incidents directly into active investigation without placing them in the intake queue.

A subtle issue appeared when checking whether work resumes after closure. Writing the rule with the standard next operator,

$$\mathbf{G}(\text{completed_closed} \rightarrow \mathbf{X} \mathbf{G} \neg \text{accepted_in_progress}),$$

gave a pass rate of only 25%. In finite trace semantics, the strong next operator $\mathbf{X}$ is always false at the final position of a trace because there is no following event. Because most tickets naturally end on `Completed+Closed`, evaluating $\mathbf{X}$ on the closing event failed on almost every completed case. Replacing the next step with $\mathbf{F}$ eliminated this end of trace issue, revealing the actual 98.4% compliance rate.

7 Comparison across logs

To compare the three logs on a unified scale, I aggregate four normalized components into a composite health score: completion rate, share of first occurrence events (measuring absence of rework), complement of the variant ratio, and a duration index benchmarked against a standard time horizon (Table 5).

Log	Score	Grade	Completion	Rework	Standardisation	Throughput
closed_problems	65.9	D	1.000	0.655	0.780	0.000
incidents	74.4	C	0.999	0.481	0.698	0.748
open_problems	70.0	D	0.435	0.746	0.778	0.942

Table 5: Composite health scores and metric components across the three logs.

While case completion is consistently high across all logs, rework constitutes a severe drag on the incident process: fewer than half of recorded incident events represent the first occurrence of their activity within a case (0.481). Repetitive loops and handovers drive the majority of operational volume.

8 Implementation

The analytical pipeline is packaged as a Python library (`volvo`) managed via `pyproject.toml` and built on Python 3.12, `pm4py`, `flloat`, `pandas`, and `Gradio`. The computational engine is strictly stateless: discovery, conformance, verification, and duration profiling routines are pure functions consuming an immutable log bundle and returning standard data structures. Session state is confined to `Gradio`’s `gr.State`, avoiding collisions between sessions.

Log ingestion in `volvo.logs.loader.load` parses CSV and XES files against declarative schemas, preserving organizational attributes (teams, country, support tier, impact) in an `EventLogBundle`. Activity labels are derived at load time under the chosen mode (`status`, `substatus`, or composite `status_substatus`). Trace verification in `volvo.verification.ltl` maps activities bijectively to propositional symbols and evaluates LTL_f formulas in parallel via `fllloat`, reporting pass rates, counterexamples, and violation indices.

The web interface exposes discovery, token and alignment replay, temporal verification, bottleneck analytics, anomaly detection, and Markov next activity prediction. Automated tests cover the full range of handlers and metrics against the raw dataset, while `scripts/build_all.py` automates artifact and document builds.

9 Conclusions

The empirical analysis of the Volvo IT log highlights three main findings:

- **Rework dominates volume:** more than half of all recorded events are repetitions, making loop reduction the primary lever for shortening resolution cycles.
- **Queueing is routinely bypassed:** support teams consistently start work without an intake queue step, confirming widespread push to front behavior.
- **Alpha miner fails on real logs:** the high prevalence of single activity loops breaks the assumptions of the Alpha algorithm, leading to degenerate models.

A couple of practical considerations apply to these numbers. Alignments take too long to compute across seven thousand traces, so I ran them on a representative sample and relied on token replay for the full log. In addition, the analysis only sees what operators actually typed into VINST. If a technician handles a problem directly and enters multiple updates afterwards, the log records an unusual sequence that reflects documentation habits rather than real operational delay.

The dataset still contains fields that were not explored here, notably the support teams, tier levels, and country codes. Looking at how tickets bounce across teams and locations would be the natural next step to understand where the rework accumulates.